[1] Create a tuple named coordinates containing three numbers: 10.5, 20.0, and 15.2. Print the tuple, its length using len(), and its data type using type().

```
# Create the tuple
coordinates = (10.5, 20.0, 15.2)

# Print the tuple itself
print("Coordinates:", coordinates)

# Print the length of the tuple
print("Length:", len(coordinates))

# Print the data type of the tuple
print("Type:", type(coordinates))
```

```
Coordinates: (10.5, 20.0, 15.2)
Length: 3
Type: <class 'tuple'>
```

[2] Given dimensions = (50, 100, 150, 200), use positive indexing to print the second element and negative indexing to print the last element.

```
dimensions = (50, 100, 150, 200)

print(dimensions[1])   # Second element
print(dimensions[-1])  # Last element
```

```
100
200
```

[3] Unpack the tuple point = (5, 12) into two variables x and y. Print both variables on separate lines.

```
point = (5, 12)

x, y = point

print(x)
print(y)
```

```
5
12
```

[4] Given ranks = (1, 2, 3, 2, 4, 2, 5), use the .count() method to find how many times 2 appears, and .index() to find the first position of 4.

```
ranks = (1, 2, 3, 2, 4, 2, 5)

print(ranks.count(2))
print(ranks.index(4))
```

```
3
4
```

[5] Create a single-element tuple containing the string "Python". Ensure it is recognized as a tuple and not a string, then print its type.

```

```

```
single_tuple = ("Python",)

print(type(single_tuple))
```

```
<class 'tuple'>
```

[6] Write a program attempting to change the first element of colors = ("red", "green", "blue") to "yellow". Use a try-except block to catch the TypeError and print "Tuples are immutable!".

```
colors = ("red", "green", "blue")

try:
    # Attempting to change the first element
    colors[0] = "yellow"
except TypeError:
    # Catching the TypeError raised by the invalid assignment
    print("Tuples are immutable!")
```

```
Tuples are immutable!
```

[7] Create a dictionary student = {"name": "Aarav", "age": 20, "course": "Computer Science"}. Print the value associated with the "course" key directly using square brackets [].

```
student = {
    "name": "Aarav",
    "age": 20,
    "course": "Computer Science"
}

# Accessing and printing the value for the "course" key using square brackets
print(student["course"])
```

```
Computer Science
```

[8] Given the dictionary car = {"brand": "Toyota", "model": "Corolla"}, add a new key-value pair "year": 2022. Print the updated dictionary.

```
car = {"brand": "Toyota", "model": "Corolla"}

# Adding the new key-value pair
car["year"] = 2022

# Printing the updated dictionary
print(car)
```

```
{'brand': 'Toyota', 'model': 'Corolla', 'year': 2022}
```

[9] Modify the value of "price" in product = {"item": "Laptop", "price": 45000} to 42000. Print the updated dictionary.

```
product = {"item": "Laptop", "price": 45000}

# Modifying the value associated with the "price" key
product["price"] = 42000

# Printing the updated dictionary
print(product)
```

```
{'item': 'Laptop', 'price': 42000}
```

[10] Given profile = {"username": "coder123"}, use an if-else statement with the in operator to check if "email" exists as a key. If present, print its value; otherwise, print "Not Provided".

```
profile = {"username": "coder123"}

# Checking if "email" exists as a key using the 'in' operator
if "email" in profile:
    print(profile["email"])
else:
    print("Not Provided")
```

```
Not Provided
```

[11] Given inventory = {"apples": 10, "bananas": 5, "oranges": 8}, iterate directly over the dictionary using a for loop to print each key along with its value using inventory[key].

```
inventory = {"apples": 10, "bananas": 5, "oranges": 8}

# Iterating directly over the dictionary
for key in inventory:
    print(f"{key}: {inventory[key]}")
```

```
apples: 10
bananas: 5
oranges: 8
```

[12] Given a dictionary user = {"name": "Siddharth", "role": "Admin"}, check if the key "email" exists in the dictionary using the in operator and print True or False.

```
user = {"name": "Siddharth", "role": "Admin"}

# Checking if "email" exists in the dictionary and printing the boolean result
print("email" in user)
```

```
False
```

[13] Use the .update() method to merge dict2 = {"c": 30, "d": 40} into dict1 = {"a": 10, "b": 20}. Print the updated dict1.

```
dict1 = {"a": 10, "b": 20}
dict2 = {"c": 30, "d": 40}

# Merging dict2 into dict1
dict1.update(dict2)

# Printing the updated dict1
print(dict1)
```

```
{'a': 10, 'b': 20, 'c': 30, 'd': 40}
```

[14] Given person = {"name": "Riya", "age": 25, "city": "Mumbai"}, use the .pop() method to remove "city" and print the popped value as well as the remaining dictionary.

```
person = {"name": "Riya", "age": 25, "city": "Mumbai"}

# Removing "city" and storing the popped value
popped_value = person.pop("city")

# Printing the popped value and the remaining dictionary
print("Popped value:", popped_value)
print("Remaining dictionary:", person)
```

```
Popped value: Mumbai
Remaining dictionary: {'name': 'Riya', 'age': 25}
```

[15] Given session = {"user": "admin", "token": "xyz123", "status": "active"}, use the .popitem() method to remove and return the last inserted key-value pair.

```
session = {"user": "admin", "token": "xyz123", "status": "active"}

# Removing and returning the last inserted key-value pair
removed_item = session.popitem()

# Printing the removed item and the updated dictionary
print("Removed item:", removed_item)
print("Updated dictionary:", session)
```

```
Removed item: ('status', 'active')
Updated dictionary: {'user': 'admin', 'token': 'xyz123'}
```

[16] Given settings = {"theme": "dark", "volume": 80}, use the del keyword to remove the "volume" key. Print the remaining dictionary.

```
settings = {"theme": "dark", "volume": 80}

# Removing the "volume" key using the del keyword
del settings["volume"]

# Printing the remaining dictionary
print(settings)
```

```
{'theme': 'dark'}
```

[17] Given data = {"a": 1, "b": 2, "c": 3}, create a duplicate copy using .copy(). Clear all items from the original dictionary using .clear(). Verify that the copy remains intact.

```
data = {"a": 1, "b": 2, "c": 3}

# Creating a duplicate copy
data_copy = data.copy()

# Clearing all items from the original dictionary
data.clear()

# Verifying the original is empty and the copy remains intact
print("Original dictionary:", data)
print("Copied dictionary:", data_copy)
```

```
Original dictionary: {}
Copied dictionary: {'a': 1, 'b': 2, 'c': 3}
```

[18] Write a program that converts a tuple num_tuple = (10, 20, 30) into a list, appends 40 to the list, converts it back into a tuple, and prints the updated tuple.

```
num_tuple = (10, 20, 30)

# Converting the tuple into a list
num_list = list(num_tuple)

# Appending 40 to the list
num_list.append(40)

# Converting the list back into a tuple
```

```
num_tuple = tuple(num_list)

# Printing the updated tuple
print(num_tuple)
```

```
(10, 20, 30, 40)
```

[19] Take a string from the user. Write a program that uses a loop and a dictionary to count the frequency of each character in the string. (e.g., "apple" → {'a': 1, 'p': 2, 'l': 1, 'e': 1}).

```
# Take a string input from the user
text = input("Enter a string: ")

# Initialize an empty dictionary to store character frequencies
frequency_dict = {}

# Iterate over each character in the string using a loop
for char in text:
    # If the character is already a key in the dictionary, increment its count
    if char in frequency_dict:
        frequency_dict[char] += 1
    # If it's not in the dictionary, add it with a count of 1
    else:
        frequency_dict[char] = 1

# Print the resulting dictionary
print(frequency_dict)
```

```
Enter a string: 6
{'6': 1}
```

[20] Given a dictionary of student marks marks = {"Alice": 85, "Bob": 42, "Charlie": 78, "David": 35}, iterate through the student names and print whether each student "Passed" (mark >= 40) or "Failed".

```
marks = {"Alice": 85, "Bob": 42, "Charlie": 78, "David": 35}

# Iterate through the dictionary using .items()
for student, mark in marks.items():
    if mark >= 40:
        print(f"{student}: Passed")
    else:
        print(f"{student}: Failed")
```

```
Alice: Passed
Bob: Passed
Charlie: Passed
David: Failed
```

[21] The Immutable GPS Log A telemetry system records continuous location coordinates as tuples inside a list: route = [(18.52, 73.85), (19.07, 72.87), (12.97, 77.59)].

- Write a program that loops through route.
- Unpack each coordinate tuple into latitude and longitude.
- Print a formatted log string: "Checkpoint X -> Latitude: Y, Longitude: Z".
- Calculate the average latitude across all checkpoints using a loop.

```
route = [(18.52, 73.85), (19.07, 72.87), (12.97, 77.59)]

total_latitude = 0

# Loop through the route, keeping track of the checkpoint number
```

```
for index, (latitude, longitude) in enumerate(route, start=1):
    # Print the formatted log string
    print(f"Checkpoint {index} -> Latitude: {latitude}, Longitude: {longitude}")

    # Add the current latitude to the total for averaging later
    total_latitude += latitude

# Calculate the average latitude
average_latitude = total_latitude / len(route)

print(f"\nAverage Latitude: {average_latitude:.4f}")
```

```
Checkpoint 1 -> Latitude: 18.52, Longitude: 73.85
Checkpoint 2 -> Latitude: 19.07, Longitude: 72.87
Checkpoint 3 -> Latitude: 12.97, Longitude: 77.59

Average Latitude: 16.8533
```

[22] The E-Commerce Shopping Cart Checkout

An online store tracks items selected by a user in a dictionary where the key is the product name and the value is its price: cart = {"Shirt": 800, "Jeans": 1500, "Shoes": 2200, "Hat": 300}.

- Ask the user if they have a promo code.
- If the promo code is "DISCOUNT10", apply a 10% discount to all items by updating the values in the dictionary.
- Calculate the final total bill by looping through the keys of cart and adding up cart[key].
- If the total bill exceeds 3000, print "Eligible for Free Shipping!", otherwise print "Standard Shipping Applied".

```
cart = {"Shirt": 800, "Jeans": 1500, "Shoes": 2200, "Hat": 300}

# Ask the user if they have a promo code
promo_code = input("Enter promo code (press Enter to skip): ")

# Apply a 10% discount if the code matches
if promo_code == "DISCOUNT10":
    print("Promo code applied! 10% discount on all items.")
    for item in cart:
        cart[item] = cart[item] * 0.9  # Deduct 10% from the price

# Calculate the final total bill by looping through keys
total_bill = 0
for key in cart:
    total_bill += cart[key]

# Display the total bill
print(f"Total Bill: ₹{total_bill:.2f}")

# Determine shipping eligibility
if total_bill > 3000:
    print("Eligible for Free Shipping!")
else:
    print("Standard Shipping Applied")
```

```
Enter promo code (press Enter to skip): DISCOUNT
Total Bill: ₹4800.00
Eligible for Free Shipping!
```

[23] The Bank Account Manager (Dictionary + Nested Keys) A bank branch stores customer data using account numbers as keys: accounts = {1001: {"name": "Priya", "balance": 5000}, 1002: {"name": "Rahul", "balance": 12000}}.

- Ask the user to enter their account number.
- Use the in operator to check if the account exists.
- If it exists, ask whether they want to "deposit" or "withdraw", along with the amount.

- Update the "balance" key inside the nested dictionary directly using accounts[acc_num]["balance"].
- If a withdrawal exceeds the available balance, cancel the transaction and print "Insufficient Funds".

```
accounts = {
    1001: {"name": "Priya", "balance": 5000},
    1002: {"name": "Rahul", "balance": 12000}
}

# Ask the user to enter their account number
# Converting input to an integer because the dictionary keys are integers
acc_num = int(input("Enter your account number: "))

# Use the 'in' operator to check if the account exists
if acc_num in accounts:
    print(f"Welcome, {accounts[acc_num]['name']}!")

    # Ask for the transaction type and amount
    action = input("Do you want to 'deposit' or 'withdraw'? ").strip().lower()
    amount = float(input("Enter the amount: "))

    if action == "deposit":
        # Update the balance directly
        accounts[acc_num]["balance"] += amount
        print(f"Deposit successful! Your new balance is ₹{accounts[acc_num]['balance']}")

    elif action == "withdraw":
        # Check if withdrawal exceeds the available balance
        if amount > accounts[acc_num]["balance"]:
            print("Insufficient Funds. Transaction cancelled.")
        else:
            # Update the balance directly
            accounts[acc_num]["balance"] -= amount
            print(f"Withdrawal successful! Your new balance is ₹{accounts[acc_num]['balance']}")

    else:
        print("Invalid action selected.")

else:
    print("Account not found.")
```

```
Enter your account number: 1002
Welcome, Rahul!
Do you want to 'deposit' or 'withdraw'? withdraw
Enter the amount: 100
Withdrawal successful! Your new balance is ₹11900.0
```

[24] The Hotel Room Booking Reset A boutique hotel manages room statuses using a dictionary: rooms = {101: "Occupied", 102: "Vacant", 103: "Occupied", 104: "Under Maintenance", 105: "Occupied"}.

- Write a program that loops through the room numbers (keys in rooms).
- For every room where rooms[room_number] == "Occupied", update its status to "Needs Cleaning".
- Maintain a counter variable to count how many total rooms were changed to "Needs Cleaning".
- Print the updated rooms dictionary and the total count.

```
rooms = {
    101: "Occupied",
    102: "Vacant",
    103: "Occupied",
    104: "Under Maintenance",
    105: "Occupied"
}

# Counter to keep track of how many rooms were changed
```

```
cleaning_count = 0

# Loop through the room numbers (keys)
for room_number in rooms:
    # Check if the room is occupied
    if rooms[room_number] == "Occupied":
        # Update the status and increment the counter
        rooms[room_number] = "Needs Cleaning"
        cleaning_count += 1

# Print the updated dictionary and the total count
print("Updated rooms:", rooms)
print("Total rooms changed to 'Needs Cleaning':", cleaning_count)
```

```
Updated rooms: {101: 'Needs Cleaning', 102: 'Vacant', 103: 'Needs Cleaning', 104: 'Under Maintenance', 105: 'Ne
Total rooms changed to 'Needs Cleaning': 3
```

[25] The Contact Book Search & Purge

You are designing the backend for a mobile phonebook app: contacts = {"Amit": "9876543210", "Neha": "9123456789", "Suresh": "9988776655"}.

- Ask the user for a contact name to search.
- If the name exists in contacts (using in), ask if they want to update the phone number or delete the contact.
- If they choose update, take a new number input and update the entry using .update().
- If they choose delete, use .pop() to remove the key and print "Contact deleted successfully."
- If the name does not exist, ask if they want to add it as a new contact.

```
contacts = {"Amit": "9876543210", "Neha": "9123456789", "Suresh": "9988776655"}

# Ask the user for a contact name to search
search_name = input("Enter contact name to search: ")

# Check if the name exists in the dictionary
if search_name in contacts:
    print(f"Contact found: {search_name} - {contacts[search_name]}")
    action = input("Do you want to 'update' or 'delete' this contact? ").strip().lower()

    if action == "update":
        new_number = input("Enter the new phone number: ")
        # Using .update() to modify the entry
        contacts.update({search_name: new_number})
        print("Contact updated successfully.")

    elif action == "delete":
        # Using .pop() to remove the key
        contacts.pop(search_name)
        print("Contact deleted successfully.")

    else:
        print("Invalid choice.")

else:
    # If the name does not exist, ask to add it
    add_choice = input("Contact not found. Do you want to add it as a new contact? (yes/no): ").strip().lower(

    if add_choice == "yes":
        new_number = input("Enter the phone number: ")
        # Adding the new contact
        contacts[search_name] = new_number
        print("Contact added successfully.")
    else:
        print("No changes made.")
```

```
# Display the final state of the contacts
print("\nFinal Phonebook:", contacts)
```

```
Enter contact name to search: Amit
Contact found: Amit - 9876543210
Do you want to 'update' or 'delete' this contact? update
Enter the new phone number: 0000000000
Contact updated successfully.

Final Phonebook: {'Amit': '0000000000', 'Neha': '9123456789', 'Suresh': '9988776655'}
```

[26] The Library Inventory & Return System A school library tracks available copies of books: library = {"Python Basics": 4, "Data Structures": 2, "Web Dev": 0}.

● A student wants to borrow "Web Dev". Check if the book exists in library using in and if its count library["Web Dev"] is greater than 0.

● If the count is 0, print "Book currently out of stock."

● If a student returns 2 copies of "Python Basics", update the inventory count by adding 2 to library["Python Basics"].

● Automatically delete any book entry from library using del if its inventory count ever drops to 0.

```
library = {"Python Basics": 4, "Data Structures": 2, "Web Dev": 0}

# 1. A student wants to borrow "Web Dev"
book_to_borrow = "Web Dev"

if book_to_borrow in library:
    if library[book_to_borrow] > 0:
        print(f"Successfully borrowed '{book_to_borrow}'.")
        library[book_to_borrow] -= 1
    else:
        print("Book currently out of stock.")
else:
    print("Book not found in the library.")

# 2. A student returns 2 copies of "Python Basics"
print("\nProcessing return: 2 copies of 'Python Basics'")
if "Python Basics" in library:
    library["Python Basics"] += 2
else:
    library["Python Basics"] = 2

# 3. Automatically delete any book entry if its inventory count drops to 0
# We use a list to store keys to delete because modifying a dictionary while iterating over it causes an error
books_to_remove = []
for book in library:
    if library[book] == 0:
        books_to_remove.append(book)

# Delete the identified books using the 'del' keyword
for book in books_to_remove:
    del library[book]
    print(f"Removed '{book}' from inventory as count reached 0.")

# Display the final updated library inventory
print("\nFinal Library Inventory:", library)
```

```
Book currently out of stock.

Processing return: 2 copies of 'Python Basics'
Removed 'Web Dev' from inventory as count reached 0.

Final Library Inventory: {'Python Basics': 6, 'Data Structures': 2}
```

[27] The Voting System Duplicate Filter (Tuple Keys) An election poll uses voter records where keys are tuples containing (voter_id, region_code) to prevent duplicate voting: voter_registry = {(101, "WEST"): "Voted", (102, "EAST"): "Voted"}.

- A new voter arrives with voter_id = 101 and region_code = "WEST".
- Create a tuple current_voter = (101, "WEST").
- Check if current_voter exists in voter_registry using the in operator.
- If it exists, print "Error: Voter has already cast a vote!"
- If it does not exist, add current_voter to voter_registry with value "Voted" and print "Vote recorded successfully!"

```
# Initial voter registry with tuple keys
voter_registry = {
    (101, "WEST"): "Voted",
    (102, "EAST"): "Voted"
}

# A new voter arrives
voter_id = 101
region_code = "WEST"

# Create a tuple for the current voter
current_voter = (voter_id, region_code)

# Check if the current voter exists in the registry
if current_voter in voter_registry:
    print("Error: Voter has already cast a vote!")
else:
    # Add the current voter to the registry
    voter_registry[current_voter] = "Voted"
    print("Vote recorded successfully!")

# Optional: Print the updated registry to verify
# print(voter_registry)
```

```
Error: Voter has already cast a vote!
```

[28] The Employee Shift Expirations (popitem() simulation) A warehouse assigns task queues to workers using a dictionary ordered by shift priority: task_queue = {"Task1": "Unload Truck", "Task2": "Inspect Boxes", "Task3": "Label Packages"}.

- Write a while loop that processes tasks one by one from the end of the queue using .popitem().
- In each iteration, unpack the popped tuple into task_id and task_desc, then print "Processing and removing: [task_id] -> [task_desc]".
- Stop the loop when len(task_queue) == 0 and print "All shift tasks completed!"

```
task_queue = {
    "Task1": "Unload Truck",
    "Task2": "Inspect Boxes",
    "Task3": "Label Packages"
}

# Run the loop as long as the dictionary is not empty
while len(task_queue) > 0:
    # Pop and unpack the last inserted key-value pair
    task_id, task_desc = task_queue.popitem()

    # Print the processed task
    print(f"Processing and removing: {task_id} -> {task_desc}")

# Print the final message when the loop finishes
print("All shift tasks completed!")
```

```
Processing and removing: Task3 -> Label Packages
Processing and removing: Task2 -> Inspect Boxes
Processing and removing: Task1 -> Unload Truck
All shift tasks completed!
```

[29] The Gradebook Merging & Cleaning Strategy Two academic branches merge their mid-term score sheets: branch1 = {"StudentA": 78, "StudentB": 85, "StudentC": 90} branch2 = {"StudentB": 88, "StudentD": 92} (Note: StudentB took tests in both branches).

● Create a copy of branch1 named final_gradebook.

● Loop through the keys of branch2. If a student already exists in final_gradebook, update their score only if the new score in branch2 is higher.

● If the student does not exist in final_gradebook, add them.

● Print the merged final_gradebook.

```
branch1 = {"StudentA": 78, "StudentB": 85, "StudentC": 90}
branch2 = {"StudentB": 88, "StudentD": 92}

# Create a copy of branch1 named final_gradebook
final_gradebook = branch1.copy()

# Loop through the keys of branch2
for student in branch2:
    # If the student already exists, update only if the new score is higher
    if student in final_gradebook:
        if branch2[student] > final_gradebook[student]:
            final_gradebook[student] = branch2[student]
    # If the student does not exist, add them
    else:
        final_gradebook[student] = branch2[student]

# Print the merged final gradebook
print("Final Gradebook:", final_gradebook)
```

```
Final Gradebook: {'StudentA': 78, 'StudentB': 88, 'StudentC': 90, 'StudentD': 92}
```

[30] The Smart Pantry Restock Alert A smart refrigerator tracks grocery levels in percentages: pantry = {"Milk": 15, "Eggs": 80, "Butter": 5, "Cheese": 50, "Juice": 10}.

● Create two empty lists: critical_restock and sufficient_stock.

● Iterate through the items in pantry using a for loop over its keys.

● If pantry[item] is below 20%, add a tuple (item, pantry[item]) to critical_restock.

● If pantry[item] is 20% or higher, append the item name to sufficient_stock.

- Print both lists clearly formatted for the user's shopping display.

```
pantry = {"Milk": 15, "Eggs": 80, "Butter": 5, "Cheese": 50, "Juice": 10}

# Create empty lists for categorization
critical_restock = []
sufficient_stock = []

# Iterate through the keys in the pantry dictionary
for item in pantry:
    # Check if stock level is below 20%
    if pantry[item] < 20:
        # Add as a tuple (item, stock_level)
        critical_restock.append((item, pantry[item]))
    else:
        # Add only the item name
        sufficient_stock.append(item)

# Print the categorized lists with clear formatting
print("SMART PANTRY SHOPPING DISPLAY")
print("============================")

print("\nCRITICAL RESTOCK NEEDED (Under 20%):")
if critical_restock:
    for item, level in critical_restock:
        print(f"  * {item} (Current level: {level}%)")
else:
    print("  * All good! No critical items.")

print("\nSUFFICIENT STOCK (20% or more):")
if sufficient_stock:
    for item in sufficient_stock:
        print(f"  * {item}")
else:
    print("  * None.")
```

```
SMART PANTRY SHOPPING DISPLAY
============================

CRITICAL RESTOCK NEEDED (Under 20%):
  * Milk (Current level: 15%)
  * Butter (Current level: 5%)
  * Juice (Current level: 10%)
```